Lab 3.1: Creating a Dynamic Menu

In this lab, you will add items to a menu dynamically.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 3.1 Demonstration.")

Estimated time to complete this lab: **20 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch03\Lab3.1\Ex01.

The solution code for this lab is located in the folder \Labs\Ch03\Lab3.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Consolidate command handlers.
- Add menu items dynamically.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter:

Exercise 1: Creating a Dynamic Menu

In this exercise, you will add items to a menu dynamically.

59 Exercise 1: Creating a Dynamic Menu

The code that forms the starting point for this exercise is in \Labs\Ch03\Lab3.1\Ex01.

In this exercise, you will add items to a menu dynamically.

When a user selects the menu item **Extra Colors**, your application will add the items **Cyan**, **Purple**, and **Yellow** to the **Colors** menu.

▶ Consolidate the command range

☐ 1. Open the file DynaView.cpp and edit the message map of the **View** class to map the commands with their respective member functions as follows:

```
ON_COMMAND(ID_OPTIONS_EXTRACOLORS, OnOptionsExtracolors)
ON_COMMAND(ID_OPTIONS_STANDARDCOLORS, OnOptionsStandardcolors)
ON_COMMAND_RANGE(ID_COLORS_CYAN, ID_COLORS_YELLOW, OnColors)
ON_COMMAND_RANGE(ID_COLORS_BLACK, ID_COLORS_BLUE, OnColors)
```

☐ 2. Open the file DynaView.h and add the following code to the section under generated message map functions:

```
afx_msg void OnOptionsExtracolors();
afx_msg void OnColors(UINT nID);
afx_msg void OnUpdateColors(CCmdUI* pCmdUI);
afx_msg void OnUpdateOptions(CCmdUI* pCmdUI);
afx_msg void OnOptionsStandardcolors();
```

▶ Add menu items dynamically

☐ 1. Open DynaView.cpp and write the following code to add menu items to the **Colors** menu when the item **Extra Colors** is selected:

```
void CMenusDynamicView::OnOptionsExtracolors()
{
    CMenu *pAddinMenu, *pTopMenu;

    // To append to the Colors menu, a pointer is required.
    // Obtain a pointer to the top-level menu.
    pTopMenu = AfxGetMainWnd()->GetMenu();

    // Colors is the 3rd menu, but that's #2 in a 0-based system.
    pAddinMenu = pTopMenu->GetSubMenu(2);
    ASSERT(pAddinMenu != NULL);

    // First, add a separator to separate the default menus
    // from the dynamic menus.
    pAddinMenu->AppendMenu(MF_SEPARATOR);
```

(Code continues on following page.)

Notes

```
    // Append the 3 menu items. They will generate consecutive
    // command IDs.
    CString prompt;
    for (int i = 0; i < 3; i++)
    {
        prompt.LoadString(ID_COLORS_CYAN + i);
        pAddinMenu->AppendMenu(MF_STRING,
                        ID_COLORS_CYAN + i, prompt);
    }
}
```

☐ 2. Add strings to the string table to store the following menu item labels.

ID	Caption
ID_COLORS_CYAN	&Cyan
ID_COLORS_PURPLE	&Purple
ID_COLORS_YELLOW	&Yellow

▶ Implement the OnColors function

- You now need to write a command handler for the dynamic menu you just created. Write the following code to handle the commands that are generated by the additional menus:

```
void CMenusDynamicView::OnColors(UINT nID)
{
    CMenusDynamicDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    pDoc->SetColor(IDtoColorRef(nID));
    Invalidate();
}
```

▶ Implement the OnOptionsStandardcolors function

- When the user selects the menu item **Standard Colors**, you need to set the menu back to the standard options that were available before the menu items were added. Write code for the **OnOptionsStandardcolors** function as follows:

```
void CMenusDynamicView::OnOptionsStandardcolors()
{
    CMenu *pAddinMenu, *pTopMenu;

    // To remove items from the Colors menu, a pointer is required
    // Obtain a pointer to the top-level menu.
    pTopMenu = AfxGetMainWnd()->GetMenu();

    // Colors is the 3rd menu, but that's #2 in a 0-based system.
    pAddinMenu = pTopMenu->GetSubMenu(2);
```

(Code continues on following page.)

Notes

```
        int i = pAddinMenu->GetMenuItemCount();

        // Remove the additions to the colors menu.
        // We want to leave only 4 menu items.
        i--;                // adjust to 0-based.
        while(i > 3)
        {
            pAddinMenu->RemoveMenu(i, MF_BYPOSITION);
            i--;
        }

        // Finally, set the color to black and force a redraw;
        CMenusDynamicDoc* pDoc = GetDocument();
        pDoc->SetColor(BLACK);
        Invalidate();
    }
```

▶ **Add update handlers**

☐ 1. Edit the view's message map to include update handlers for the menus you have added:

```
ON_UPDATE_COMMAND_UI_RANGE(ID_COLORS_CYAN, ID_COLORS_YELLOW,
OnUpdateColors)
ON_UPDATE_COMMAND_UI_RANGE(ID_COLORS_BLACK, ID_COLORS_BLUE,
OnUpdateColors)
ON_UPDATE_COMMAND_UI(ID_OPTIONS_EXTRACOLORS, OnUpdateOptions)
ON_UPDATE_COMMAND_UI(ID_OPTIONS_STANDARDCOLORS, OnUpdateOptions)
```

☐ 2. Write the code for the **OnUpdateColors** handler to check the selected color in the **Colors** menu:

```
void CMenusDynamicView::OnUpdateColors(CCmdUI* pCmdUI)
{
    CMenusDynamicDoc* pDoc = GetDocument();
    ASSERT_VALID(pDoc);

    pCmdUI->SetCheck(pDoc->GetColor() == IDtoColorRef(pCmdUI->m_nID));
}
```

Notes

☐ 3. Write the code for the **OnUpdateOptions** handler to check the selected menu item in the **Options** menu:

```
void CMenusDynamicView::OnUpdateOptions(CCmdUI* pCmdUI)
{
    CMenu *pAddinMenu, *pTopMenu;
    pTopMenu = AfxGetMainWnd()->GetMenu();
    pAddinMenu = pTopMenu->GetSubMenu(2);

    switch (pCmdUI->m_nID)
    {
        case ID_OPTIONS_EXTRACOLORS:
            pCmdUI->Enable(pAddinMenu->GetMenuItemCount() == 4);
            break;
        case ID_OPTIONS_STANDARDCOLORS:
            pCmdUI->Enable(pAddinMenu->GetMenuItemCount() != 4);
    }
}
```

☐ 4. Save DynaView.cpp. Build Dynamenu and run it.

The solution code for this exercise is located in the folder \Labs\Ch03\Lab3.1\Ex01\Solution.

Notes